COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE031	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 17, 2026	**DATE SUBMITTED:**	Oct 24, 2026
NAME:	ANDREW DOLOSO JR. RALPH FLORENZ DUE?AS JAREL DAVE EGAR	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

DOLOSO	
Task 1	**Task 1:** Define the Purpose Write a short paragraph (3–5 sentences) describing the purpose of your dashboard. • The academic dashboard is used to analyze multiple perspectives of the analysis done by the head of departments to teachers, and students to assess education results. The Department Head Dashboard analyzes overall performance of each subject, average, and scores for each course such as DSA, DE, OOP, and VDA. On the other hand, the student and teacher dashboards help in analyzing progress personally, by comparing the scores of each student compared to class average.

Task 2

```python
# STUDENT DASHBOARD

import pandas as pd
import plotly.graph_objects as go

# 1. Prepare data
# Supports either "Score" or "Grade"
if "Score" not in df.columns and "Grade" in df.columns:
    df.rename(columns={"Grade": "Score"}, inplace=True)

df.columns = df.columns.str.strip()

# 2. Aggregate scores per student per subject
student_df = df.groupby(
    ["Student", "Subject"], as_index=False
)["Score"].mean()

# 3. Compute class average per subject
class_avg = df.groupby(
    "Subject", as_index=False
)["Score"].mean().rename(
    columns={"Score": "Class Average"}
)

# 4. Select two students
student_names = sorted(df["Student"].unique())

student_1 = student_names[0]
student_2 = student_names[1] if len(student_names) > 1 else student_names[0]

# 5. Extract each student's scores
s1_scores = student_df[
    student_df["Student"] == student_1
][["Subject", "Score"]].rename(
    columns={"Score": f"{student_1} Score"}
)

s2_scores = student_df[
    student_df["Student"] == student_2
][["Subject", "Score"]].rename(
    columns={"Score": f"{student_2} Score"}
)

# 6. Merge the data
comparison_df = pd.merge(
    class_avg, s1_scores, on="Subject", how="left"
)

comparison_df = pd.merge(
    comparison_df, s2_scores, on="Subject", how="left"
)

# 7. Create grouped bar chart
fig_student = go.Figure()

fig_student.add_trace(go.Bar(
    x=comparison_df["Subject"],
    y=comparison_df[f"{student_1} Score"],
    name=f"{student_1} Score",
    marker_color="royalblue"
))

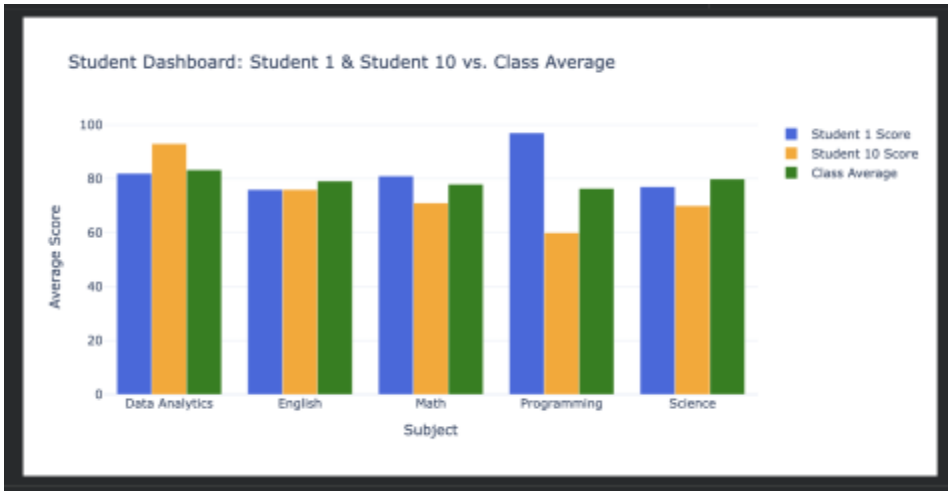
fig_student.add_trace(go.Bar(
    x=comparison_df["Subject"],
    y=comparison_df[f"{student_2} Score"],
    name=f"{student_2} Score",
    marker_color="orange"
))

fig_student.add_trace(go.Bar(
    x=comparison_df["Subject"],
    y=comparison_df["Class Average"],
    name="Class Average",
    marker_color="green"
))

# 8. Update layout
fig_student.update_layout(
    barmode="group",
    height=450,
    width=900,
    title_text=(
        f"Student Dashboard: {student_1} & "
        f"{student_2} vs. Class Average"
    ),
    xaxis_title="Subject",
    yaxis_title="Average Score",
    template="plotly_white"
)
```

COMPUTER ENGINEERING

QUEZON CITY

DOLOSO	
	 TEACHERS DASHBOARD

```python
# TEACHER DASHBOARD

import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from IPython.display import display

# 1. column names
df.columns = df.columns.str.strip()

rename_columns = {
    "Score": "Grade",
    "Student Name": "Student",
    "Attendance (%)": "Attendance"
}

df.rename(columns=rename_columns, inplace=True)

# 2. Check required columns
required = ["Student", "Grade", "Attendance"]
missing = [col for col in required if col not in df.columns]

if missing:
    raise ValueError(
        f"Missing columns: {missing}. "
        f"Available columns: {df.columns.tolist()}"
    )

# Convert grades and attendance to numeric values
df["Grade"] = pd.to_numeric(df["Grade"], errors="coerce")
df["Attendance"] = pd.to_numeric(
    df["Attendance"], errors="coerce"
)

# Remove rows with missing values in required fields
teacher_data = df.dropna(
    subset=["Student", "Grade", "Attendance"]
).copy()

# 3. Set passing threshold
threshold = 75

teacher_data["Status"] = teacher_data["Grade"].apply(
    lambda grade: "Underperforming (<75)"
    if grade < threshold else "Passing (>=75)"
)

# 4. Calculate each student's average grade
student_avg = teacher_data.groupby(
    "Student", as_index=False
)["Grade"].mean()

underperforming = student_avg[
    student_avg["Grade"] < threshold
].sort_values("Grade")

# Display summary
print("TEACHER DASHBOARD")
print("Total students:", teacher_data["Student"].nunique())
print("Underperforming students:", len(underperforming))
print("\nStudents needing academic support:")

if underperforming.empty:
    print("No students have an average grade below 75.")
else:
    display(underperforming)

# 5. Create dashboard charts
fig_teacher = make_subplots(
    rows=1, cols=2,
    subplot_titles=(
        "Underperforming Students (Average Grade <75)",
        "Grades vs. Attendance"
    ),
    specs=[[{"type": "bar"}, {"type": "scatter"}]]
)

# Bar chart of underperforming students
fig_teacher.add_trace(
    go.Bar(
        x=underperforming["Student"],
        y=underperforming["Grade"],
        marker_color="darkred",
        name="Underperforming"
    ),
    row=1, col=1
```

COMPUTER ENGINEERING

QUEZON CITY

DOLOSO

```
        row=1, col=2
    )

fig_teacher.update_xaxes(
    title_text="Student",
    row=1, col=1
)
fig_teacher.update_yaxes(
    title_text="Average Grade",
    row=1, col=1
)
fig_teacher.update_xaxes(
    title_text="Attendance (%)",
    row=1, col=2
)
fig_teacher.update_yaxes(
    title_text="Grade",
    row=1, col=2
)

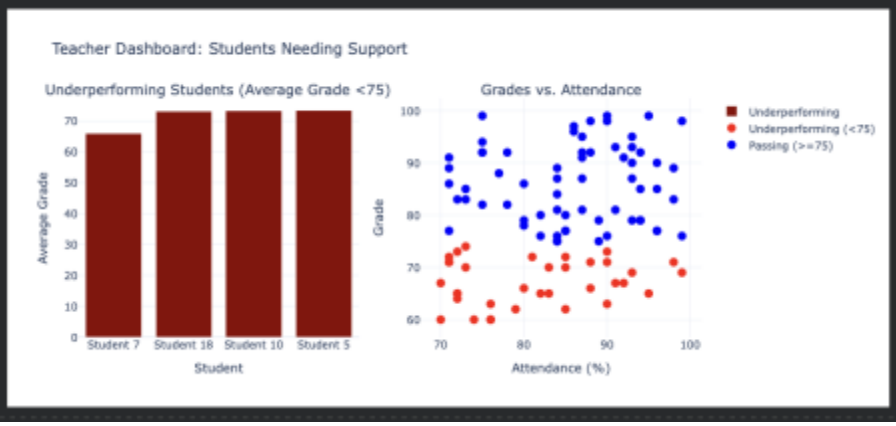
fig_teacher.update_layout(
    title_text="Teacher Dashboard: Students Needing Support",
    height=450,
    width=1000,
    template="plotly_white"
)

fig_teacher.show()
```

```
TEACHER DASHBOARD
Total students: 20
Underperforming students: 4

Students needing academic support:
```

	Student	Grade
17	Student 7	66.000000
9	Student 18	73.125000
1	Student 10	73.333333
15	Student 5	73.500000



COMPUTER ENGINEERING

QUEZON CITY

DOLOSO	
	DEPARTMENT HEAD

```python
# DEPARTMENT HEAD
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# 1. Create dataset with 8 students and 4 subjects
df = pd.DataFrame({
    "Student": [
        "Student 1", "Student 2", "Student 3", "Student 4",
        "Student 5", "Student 6", "Student 7", "Student 8"
    ],
    "Subject": ["DSA", "DSA", "DE", "DE", "OOP", "OOP", "VDA", "VDA"],
    "Score": [88, 76, 90, 82, 85, 78, 92, 86],
    "Attendance": [95, 85, 96, 90, 92, 80, 98, 94]
})

# 2. Calculate subject-level averages
subject_summary = df.groupby(
    "Subject", as_index=False
).agg(
    Average_Score=("Score", "mean"),
    Highest_Score=("Score", "max"),
    Lowest_Score=("Score", "min")
)

# Keep subjects in the requested order
subject_order = ["DSA", "DE", "OOP", "VDA"]
subject_summary["Subject"] = pd.Categorical(
    subject_summary["Subject"],
    categories=subject_order,
    ordered=True
)
subject_summary = subject_summary.sort_values("Subject")

# 3. Display summary
print("DEPARTMENT HEAD DASHBOARD")
print("=" * 35)
print("Number of Subjects:", len(subject_summary))
print("\nSUBJECT PERFORMANCE SUMMARY")
display(subject_summary.round(2))

# 4. Create dashboard charts
fig_head = make_subplots(
    rows=1, cols=2,
    subplot_titles=(
        "Average Score by Subject",
        "Highest and Lowest Scores by Subject"
    )
)

# Chart 1: Average score
fig_head.add_trace(
    go.Bar(
        x=subject_summary["Subject"],
        y=subject_summary["Average_Score"],
        marker_color="teal",
```

COMPUTER ENGINEERING

QUEZON CITY

DOLOSO

```
            y=subject_summary["Average_Score"],
            marker_color="teal",
            text=subject_summary["Average_Score"],
            textposition="auto",
            name="Average Score"
        ),
        row=1, col=1
    )

    # Chart 2: Highest and lowest scores
    fig_head.add_trace(
        go.Bar(
            x=subject_summary["Subject"],
            y=subject_summary["Highest_Score"],
            name="Highest Score",
            marker_color="royalblue"
        ),
        row=1, col=2
    )

    fig_head.add_trace(
        go.Bar(
            x=subject_summary["Subject"],
            y=subject_summary["Lowest_Score"],
            name="Lowest Score",
            marker_color="orange"
        ),
        row=1, col=2
    )

    # 5. Update layout
    fig_head.update_xaxes(title_text="Subject", row=1, col=1)
    fig_head.update_yaxes(title_text="Average Score", row=1, col=1)
    fig_head.update_xaxes(title_text="Subject", row=1, col=2)
    fig_head.update_yaxes(title_text="Score", row=1, col=2)

    fig_head.update_layout(
        title_text="Department Head Dashboard: DSA, DE, OOP, and VDA",
        barmode="group",
        height=500,
        width=1000,
        template="plotly_white"
    )

    fig_head.show()
```

COMPUTER ENGINEERING

QUEZON CITY

DOLOSO

```
DEPARTMENT HEAD DASHBOARD
==========
output actions  of Subjects: 4

SUBJECT PERFORMANCE SUMMARY
```

	Subject	Average_Score	Highest_Score	Lowest_Score
1	DSA	82.0	88	76
0	DE	86.0	90	82
2	OOP	81.5	85	78
3	VDA	89.0	92	86



DUENAS

Task 1

Task 1: Define the Purpose

Write a short paragraph (3–5 sentences) describing the purpose of your dashboard.

This dashboard is designed to track student performance using the given dataset. It features four simple charts: a bar chart showing average subject scores, a scatter plot matching attendance against grades, a box plot showing score ranges, and another bar chart for average attendance per subject. This tool helps viewers easily see problem areas so they can better support student achievement.

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

DUENAS

Task 2

```
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Load your dataset first (replace 'student_data.csv' with your actual file path/name)
# df = pd.read_csv("student_data.csv")

# Teacher
threshold = 75
df["Status"] = df["Score"].apply(lambda x: "Underperforming (<75)" if x < threshold else "Passing (>=75)")

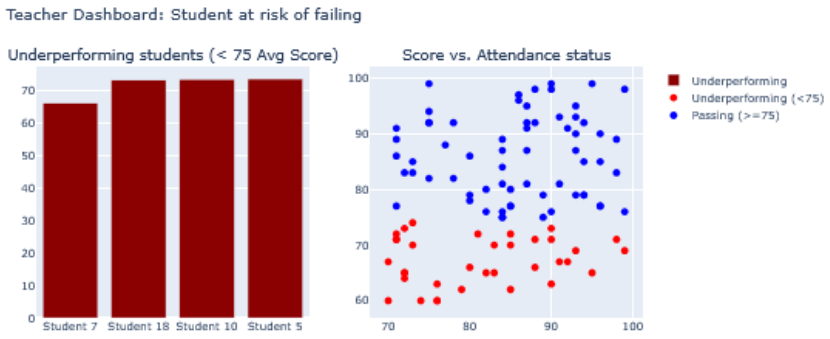
# Aggregated student average scores
student_avg = df.groupby("Student", as_index=False)["Score"].mean()
underperforming_students = student_avg[student_avg["Score"] < threshold].sort_values(by="Score")

# Create layout
fig_teacher = make_subplots(
    rows=1, cols=2,
    subplot_titles=("Underperforming students (< 75 Avg Score)", "Score vs. Attendance status"),
    specs=[[{"type": "bar"}, {"type": "scatter"}]]
)

# Bar chart of underperforming students
fig_teacher.add_trace(
    go.Bar(
        x=underperforming_students["Student"],
        y=underperforming_students["Score"],
        marker_color="DarkRed",
        name="Underperforming"
    ),
    row=1, col=1
)

# Scatter plot highlighting underperforming records in red
colors = {"Underperforming (<75)": "red", "Passing (>=75)": "Blue"}
for status, color in colors.items():
    sub_df = df[df["Status"] == status]
    fig_teacher.add_trace(
        go.Scatter(
            x=sub_df["Attendance (%)"],
            y=sub_df["Score"],
            mode="markers",
            name=status,
            marker=dict(color=color, size=8),
            hovertext=sub_df["Student"]
        ),
        row=1, col=2
    )

fig_teacher.update_layout(height=450, width=950, title_text="Teacher Dashboard: Student at risk of failing")
fig_teacher.show()
```



COMPUTER ENGINEERING

QUEZON CITY

EGAR

Task 1

Task 1: Define the Purpose

Write a short paragraph (3–5 sentences) describing the purpose of your dashboard.

The purpose of this dashboard is to show student performance based on the given data set. In the dashboard, there are 4 charts: a bar chart for the average score per subject, a scatter plot for comparing attendance with the score, a box plot for Score distribution, and another Bar chart for the average attendance per subject. This dashboard provides its reader with an easy way to identify the flaws in their system to improve the students academic success.

Task 2

Task 2: Design for User Needs

Modify the dashboard to meet specific user needs:

If your user is a teacher, highlight underperforming students.

If your user is a student, highlight their personal progress vs. class average.

If your user is a department head, summarize subject-level averages only.

```
#Teacher
threshold = 75
df["Status"] = df["Score"].apply(lambda x: "Underperforming (<75)" if x < threshold else "Passing (>=75)")

#Aggregated student average scores
student_avg = df.groupby("Student", as_index=False)["Score"].mean()
underperforming_students = student_avg[student_avg["Score"] < threshold].sort_values(by="Score")

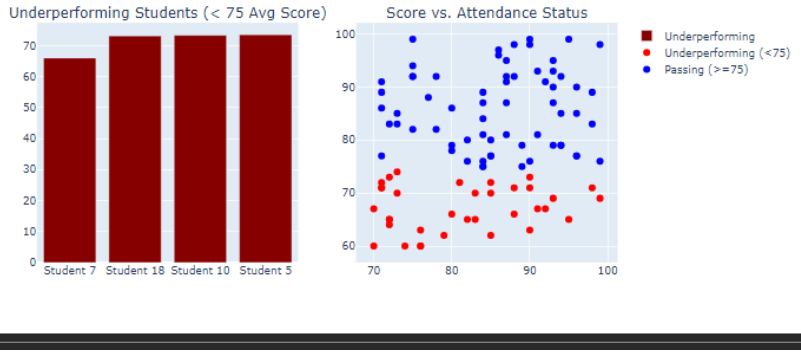
#Create layout
fig_teacher = make_subplots(rows=1, cols=2,
                            subplot_titles=("Underperforming Students (< 75 Avg Score)", "Score vs. Attendance Status"), specs=[[{"type": "bar"}, {"type": "scatter"}]])

#Bar chart of underperforming students
fig_teacher.add_trace(
    go.Bar(x=underperforming_students["Student"], y=underperforming_students["Score"], marker_color="DarkRed", name="Underperforming"),
    row=1, col=1
)

#Scatter plot highlighting underperforming records in red
colors = {"Underperforming (<75)": "red", "Passing (>=75)": "Blue"}
for status, color in colors.items():
    sub_df = df[df["Status"] == status]
    fig_teacher.add_trace(
        go.Scatter(x=sub_df["Attendance (%)"], y=sub_df["Score"], mode="markers", name=status, marker=dict(color=color, size=8), hovertext=sub_df["Student"]),
        row=1, col=2
    )

fig_teacher.update_layout(height=450, width=950, title_text="Teacher Dashboard: Student at risk of failing")
fig_teacher.show()
```

```
fig_teacher.show()
```



COMPUTER ENGINEERING

QUEZON CITY

EGAR

```python
import pandas as pd
import plotly.graph_objects as go

# 1. Aggregate scores per student per subject to avoid duplicate rows
student_df = df.groupby(["Student", "Subject"], as_index=False)["Score"].mean()

# 2. Compute class average per subject
class_avg = df.groupby("Subject", as_index=False)["Score"].mean().rename(columns={"Score": "Class Average"})

# 3. Extract scores for Student 1 and Student 2
student_1 = "Student 1"
student_2 = "Student 2"

s1_scores = student_df[student_df["Student"] == student_1][["Subject", "Score"]].rename(columns={"Score": f"{student_1} Score"})
s2_scores = student_df[student_df["Student"] == student_2][["Subject", "Score"]].rename(columns={"Score": f"{student_2} Score"})

# 4. Merge datasets together cleanly
comparison_df = pd.merge(class_avg, s1_scores, on="Subject", how="left")
comparison_df = pd.merge(comparison_df, s2_scores, on="Subject", how="left")

# 5. Create grouped bar chart
fig_student = go.Figure()

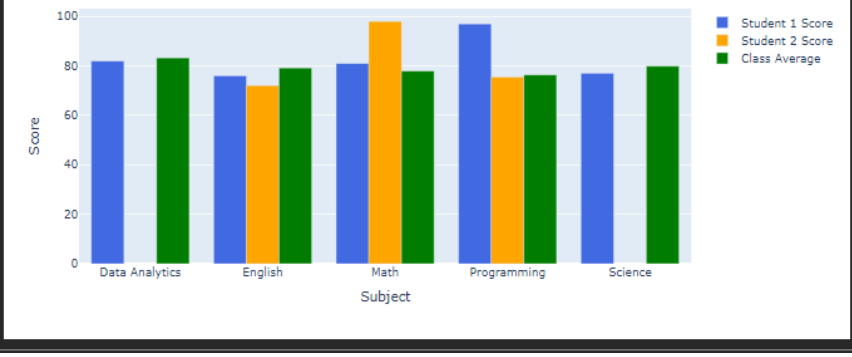
# Student 1 Bar
fig_student.add_trace(go.Bar(
    x=comparison_df["Subject"],
    y=comparison_df[f"{student_1} Score"],
    name=f"{student_1} Score",
    marker_color="royalblue"
))

# Student 2 Bar
fig_student.add_trace(go.Bar(
    x=comparison_df["Subject"],
    y=comparison_df[f"{student_2} Score"],
    name=f"{student_2} Score",
    marker_color="orange"
))

# Class Average Bar
fig_student.add_trace(go.Bar(
    x=comparison_df["Subject"],
    y=comparison_df["Class Average"],
    name="Class Average",
    marker_color="green"
))
```

```python
# Layout updates
fig_student.update_layout(
    barmode="group",
    height=450, width=900,
    title_text=f"Student Dashboard: {student_1} & {student_2} Performance vs. Class Average",
    xaxis_title="Subject",
    yaxis_title="Score"
)

fig_student.show()
```



COMPUTER ENGINEERING

QUEZON CITY

EGAR

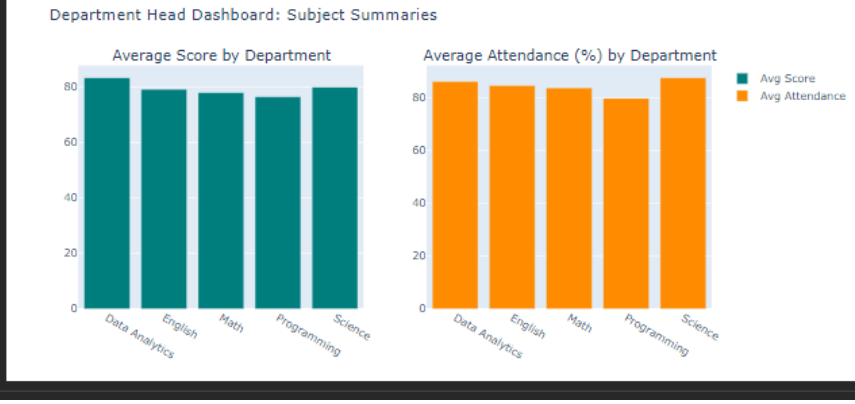
```python
# Department Head
dept_summary = df.groupby("Subject", as_index=False).agg({
    "Score": "mean",
    "Attendance (%)": "mean"
}).round(2)

#dashboard
fig_dept = make_subplots(rows=1, cols=2,
                         subplot_titles=("Average Score by Department", "Average Attendance (%) by Department"),
                         specs=[[{"type": "bar"}, {"type": "bar"}]])

fig_dept.add_trace(
    go.Bar(x=dept_summary["Subject"], y=dept_summary["Score"],
           name="Avg Score", marker_color="teal"),
    row=1, col=1
)

fig_dept.add_trace(
    go.Bar(x=dept_summary["Subject"], y=dept_summary["Attendance (%)"],
           name="Avg Attendance", marker_color="darkorange"),
    row=1, col=2
)

fig_dept.update_layout(height=450, width=950, title_text="Department Head Dashboard: Subject Summaries")
fig_dept.show()
```



T.I.P TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

SUPPLEMENTARY

```python
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

np.random.seed(42)

areas = {
    "Laboratories": (30, 0.6, 0.5),
    "Classrooms": (15, 0.9, 0.1),
    "Offices": (12, 0.8, 0.2),
    "Library": (10, 0.7, 0.6),
    "Cafeteria": (8, 0.7, 0.5),
}

colors = {
    "Laboratories": "#1f77b4",
    "Classrooms": "#2ca02c",
    "Offices": "#ff7f0e",
    "Library": "#9467bd",
    "Cafeteria": "#d62728",
}

hours = pd.date_range("2026-09-01", periods=30 * 24, freq="h")
rows = []

for area, (base, sched, wk) in areas.items():
  for t in hours:
    daytime = 1.0 if 8 <= t.hour < 18 else 0.0
    weekend = wk if t.dayofweek >= 5 else 1.0
    kWh = base * (1 + sched * daytime * weekend) * np.random.normal(1, 0.08)
    rows.append((t, area, max(kWh, 0)))

energy = pd.DataFrame(rows, columns=["Timestamp", "Area", "kWh"])
energy["Date"] = energy["Timestamp"].dt.normalize()

daily = energy.pivot_table(
    index="Date", columns="Area", values="kWh", aggfunc="sum"
)
total_by_area = energy.groupby("Area")["kWh"].sum().sort_values(ascending=False)

total = energy["kWh"].sum()
peak_hour = energy.groupby(energy["Timestamp"].dt.hour)["kWh"].sum().idxmax()
top_area = total_by_area.index[0]

print(
    f"Total consumption: {total:,.0f} kWh | "
    f"Highest consumer: {top_area} ({(total_by_area.iloc[0] / total):.0%}) | "
    f"Campus peak hour: {peak_hour}:00"
)
```

SUPPLEMENTARY

```
fig = make_subplots(
    rows=2,
    cols=2,
    specs=[
        [{"colspan": 2, "type": "xy"}, None],
        [{"type": "xy"}, {"type": "domain"}],
    ],
    row_heights=[0.55, 0.45],
    vertical_spacing=0.18,
    subplot_titles=(
        "Daily Energy Use per Area (kWh)",
        "Total Consumption per Area (kWh)",
        "Share of Campus Energy Use",
    ),
)

area_names = list(daily.columns)

for a in area_names:
  fig.add_trace(
      go.Scatter(
          x=daily.index,
          y=daily[a],
          mode="lines",
          name=a,
          line=dict(color=colors[a], width=2),
          legendgroup=a,
      ),
      row=1,
      col=1,
  )

fig.add_trace(
    go.Bar(
        x=total_by_area.index,
        y=total_by_area.values,
        marker_color=[colors[a] for a in total_by_area.index],
        text=total_by_area.round(0).astype(int),
        textposition="outside",
        showlegend=False,
    ),
    row=2,
    col=1,
)

fig.add_trace(
    go.Pie(
        labels=total_by_area.index,
        values=total_by_area.values,
        marker=dict(colors=[colors[a] for a in total_by_area.index]),
        hole=0.4,
        textinfo="percent+label",
        showlegend=False,
    ),
    row=2,
    col=2,
)

fig.update_layout(
    title_text="Smart Campus Energy Monitor Dashboard",
    height=800,
    showlegend=True,
    template="plotly_white",
)

fig.show()
```

COMPUTER ENGINEERING

QUEZON CITY

SUPPLEMENTARY



CONCLUSION / LEARNING / ANALYSIS:

Doloso: In the HOA, we learn how to develop dashboards using Python in order to present data in a systematic manner. We also knew that a dashboard should have a purpose and should provide all the necessary information to various stakeholders, which may include teachers, students, heads of departments, and facility managers. Through the use of charts, filters, and other interactive means, we were able to learn how to make data understandable and analyzable. In essence, this activity enabled us to realize that a dashboard is not just meant for presentation but for decision making as well.

Duenas: In this HOA, we learned how a dashboard's main goal shapes its design. A good dashboard is more than just a group of graphs. It should display only the details that are important to its users. We also learned how to check if a dashboard meets different people's needs, like a teacher, a student, or a department manager, by changing what each person can see. In the extra activity, we built a clickable dashboard that is clear, correct, and easy to use. This HOA showed us that making a great dashboard starts with knowing its purpose and understanding the people using it.

Egar: This Hands-On Activity taught us how to create interactive data dashboards using Python programming tools, including Pandas, NumPy, Plotly Express, and ipywidgets libraries. By analyzing sample student data and building the Smart Campus Energy Monitor, we displayed key metrics such as mean grades, attendance rates, and energy consumption through multi-chart layouts and dropdown filters. We customized our designs to meet specific user needs, including tracking underperforming students for teachers, individual progress for students, subject averages for department heads, and campus energy usage for facility managers. Ultimately, we learned that effective data dashboards must be simple, accurate, and interactive, enabling users to explore data and make better strategic, analytical, or operational decisions based on their specific goals.

COMPUTER ENGINEERING

QUEZON CITY

COLABS
DOLOSO
Doloso_Hands_On_Activity_13___Designing_a_Purpose_Driven_Data_Dashboard.ipynb
DUENAS
Duenas_Hands_On_Activity_13___Designing_a_Purpose_Driven_Data_Dashboard.ipy…
EGAR
Hands_On_Activity_13___Designing_a_Purpose_Driven_Data_Dashboard.ipynb